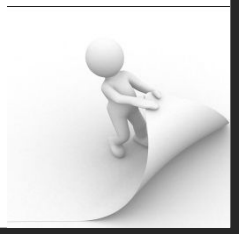
C++11新特性之四：lambda函数

张锦成 2015/10/15

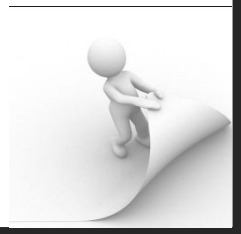


Outline

- lambda函数的一些历史
  - C++11中的lambda函数
    - lambda函数与仿函数(函数对象)
      - lambda函数与STL

1

lambda的一些历史



lambda历史

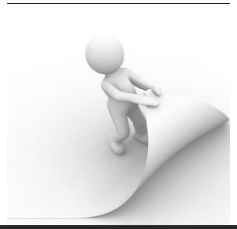
20世纪30年代：lambda演算

1958年：基于lambda演算的LISP语言

1983年：C++在Bell Lab诞生

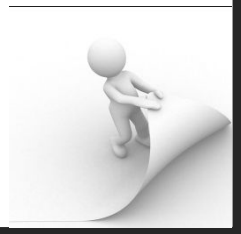
几种典型编程范型

- 面向过程编程: C/C++
- 面向对象编程: C++/Java
- 函数式编程: Haskell, Ocaml



2

C++11中的lambda函数



lambda初窥

```
#include <iostream>

using namespace std;

int main()
{
    auto sum = [](int x, int y) -> int { return x+y;};
    cout << sum(1, 2) << endl;
    return 0;
}
```

lambda函数定义

```
[capture] (parameters) mutable -> return_type {statement}
```

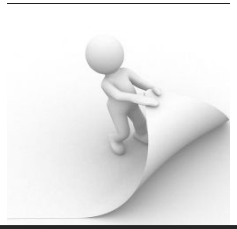
[var] 表示按值传递方式捕捉变量var
[=] 表示按值传递方式捕捉所有父作用域的变量
[&var] 表示按引用方式捕捉变量var
[&] 表示按引用方式捕捉所有父作用域的变量
[this] 表示按值传递方式捕捉当前的this指针

- 参数列表为空可省略，除非有mutable修饰符
- lambda函数默认是const函数
- mutable可以取消lambda函数的常量性，此时参数列表不可省略，即使为空
- 没有返回值或者返回值类型可推导时可以省略 -> return_type
- 捕捉列表不允许变量重复传递

```
[=, a]
[&, &this]
```

lambda函数

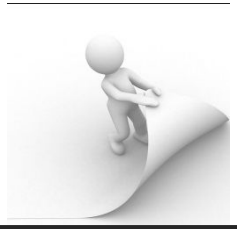
```
int a = 1;
int b = 2;

[]{};
[=] {return a+b;};
auto func1 = [&] (int c) { b = a + c;};
auto func2 = [=, &b] (int c) -> int {return b += a + c;};

cout << typeid(func2).name() << endl;
cout << func2(2) << endl;
cout << b << endl;
```



lambda函数的思考

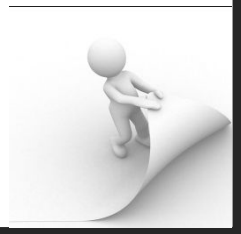
下面代码应该输出什么？

```
int j = 1;
function<void(void)> func1 = [=]() mutable {j++; cout << j << endl; };
func1();
cout << j << endl;
func1();
cout << j << endl;

function<void(void)> func2 = [&]() {j++; cout << j << endl; };
func2();
cout << j << endl;
```



3

lambda函数与仿函数

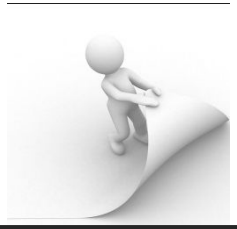


仿函数(函数对象)

- 仿函数实际上是一个类的对象
- 自定义了成员函数operator()
- 代码层面看起来和使用函数并无二样

仿函数(函数对象)

```
class _functor{
    public:
        int operator() (int x, int y) {
            return x+y;
        }
};

int girls = 3;
int boys = 4;
_functor totalChild;
cout << totalChild(girls, boys) << endl;
```



lambda函数与仿函数的联系

```
class AirportPrice{
    private:
        float _dutyFreeRate;
    public:
        AirportPrice(float rate): _dutyFreeRate(rate) {}
        float operator()(float price) {
            return price * (1 - _dutyFreeRate/100);
        }
};

int main()
{
    float taxRate = 5.5, price = 3000;
    AirportPrice szAir(taxRate);
    cout << szAir(price) << endl;

    auto szAir2 = [taxRate](float price) -> float {return price * (1-taxRate/100);};
    cout << szAir2(price) << endl;

    return 0;
}
```

lambda函数与仿函数的联系

- 仿函数是编译器实现lambda的一种方式，编译器把lambda函数转换为仿函数
- lambda函数是仿函数的“语法糖”

lambda函数

```
[tax_rate]
(float price)->float{ return price * (1 - tax_rate/100); };
```

```
class AirportPrice{
private:
    float _dutyfreerate;
public:
    AirportPrice(float rate): _dutyfreerate(rate){}

    float operator()(float price) {
        return price * (1 - _dutyfreerate/100);
    }
};
```

仿函数

图 7-1　lambda 函数及与其等价的仿函数

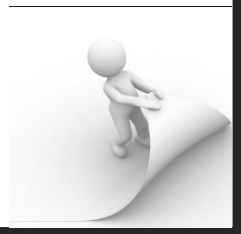
lambda函数与仿函数的联系

- lambda函数不能完全代替仿函数，主要是因为lambda捕捉列表的限制
- 设计目的：就地书写，就地使用

```
int d = 0;

int tryCapture() {
    auto ill_lambda = [d] {};
}

class HealthyFunctor{
    public:
        HealthyFunctor(int d): data(d) {}
        void operator()() const{}
    private:
        int data;
};

int tryCapture() {
    HealthyFunctor hf(d);
}
```

4

lambda函数与STL



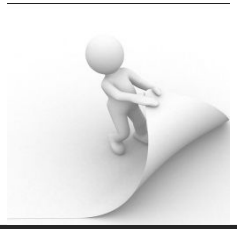
lambda与STL

- lambda函数对C++11最大的贡献在STL库中
- 使用STL的算法更加容易了

以STL的for_each算法为例

```
UnaryProc for_each(InputIterator beg, InputIterator end, UnaryProc op)
```

```
for_each(iterator begin, iterator end, Function fn){
    for(iterator i = begin; i!=end; ++i) {
        fn(*i);
    }
}
```



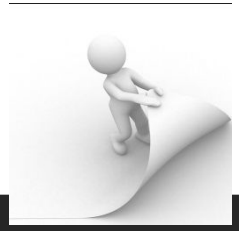
以STL的for_each算法为例

```
int total1 = 0;
void add(int i) {total1 += i;}

int main()
{
    vector<int> nums{1, 2, 3};

    for_each(nums.begin(), nums.end(), add);
    cout << total1 << endl;

    int total2 = 0;
    for_each(begin(nums), end(nums), [&](int i){total2 += i;});
    cout << total2 << endl;

    return 0;
}
```

References

1. 《深入理解C++11: C++11新特性解析与应用》第7章

2. C++11新特性: http://www.cocos.com/doc/tutorial/show?id=1120



THANK
YOU!